11753 Computational Intelligence
Master in Intelligent Systems
Universitat de les Illes Balears

Handout #2: **Feed-forward Neural Networks (FFNN)**
(graded assignment)

The samples of the datasets to be used in this assignment comprise up to 54 features corresponding to 30×30m patches of forest in the US, collected for the task of predicting each patch's cover type, i.e. the dominant species of tree. All features take integer values, some of them are boolean indicators, while the others take discrete values, and there are seven cover types, making this a multiclass classification problem:

- Elevation, Aspect, Slope: 3 integer features, global properties of the patch
- Horizontal/Vertical distance to hydrology: 2 integer features, horizontal/Vertical distance to closest surface water features
- Horizontal distance to roadways/fire points: 2 integer features, horizontal distance to nearest roadways/nearest wildfire ignition points
- Hillshade 9am, noon, 3pm: Hillshade indices at 9AM, noon and 3PM
- Wilderness area $0-3$: 4 binary features, 0 indicates absent and 1 indicates present
- Soil type $0-39$: 40 binary features, 0 indicates absent and 1 indicates present
- Cover type: target feature, labels $1-7$

Each group has its own dataset to work with. Given the group number, the dataset can be loaded as follows:

```
from pandas import read_csv
g = 1 # assuming group 1
data = read_csv('ds%02d.csv' % (g))
X = data.iloc[:,:-1].to_numpy() # samples
y = data.iloc[:,-1].to_numpy()  # target
```

Listing 1: Loading of the dataset.

Now, to address the multi-class classification problem described above, you are supposed to find a **suitable neural network**. You should try with up to 3 hidden layers, different amounts of neurons in each layer, several loss functions and optimizers. You can also incorporate the mechanisms that have been discussed in the classroom, such as dropout, a learning rate scheduler, etc.

To find an adequate configuration, split the dataset in a *training set* and a *test set*, and use the *accuracy* for the *test set* to measure the performance of each case and make your design decisions accordingly (as you can check, there is no class imbalance in the dataset, and hence the accuracy can be used as a reliable indicator of performance).

Although it is not compulsory, you can try with the following suboptimal protocol:

T1. Start with a **basic configuration**: a single hidden layer network with enough neurons (greater than the input data dimensionality) and suitable activation functions, an adequate optimizer and a reasonable loss function.

For training, set a fixed learning rate, a reasonable batch size and choose enough epochs to let the training converge. Tune the learning rate until you achieve convergence and a reasonable performance for the validation set. You can start with a small learning rate, e.g. 10^{-3}, and start increasing it until you observe oscillations in the values of the loss function over the epochs.

T2. Try (an) **alternative optimizer**(s) and keep the one that results in the best performance.

T3. Consider alternative **activation functions in the hidden layer** and keep the one that results in the best performance.

T4. Switch to **other loss function**(s) to check whether the performance level increases and keep the best.

T5. Add a **second hidden layer** with a reasonable number of neurons (usually lower than the first hidden layer) and check whether a performance gain is obtained. If that is the case, keep the second layer.

T6. In case you have added a second hidden layer, try with a **third hidden layer** with a reasonable number of neurons to increase even more the network performance. Keep that third layer if that is the case.

T7. Try with **other batch sizes**. You should observe that the training time also changes. Adopt the batch size leading to highest performance.

T8. Consider adopting a **dynamic learning rate**.

Once you have obtained a suitable configuration for your network:

T9. Measure its performance using **5-fold cross validation**: a) randomly split the dataset in 5 folds, b) train with four folds and test with the remaining fold until all folds have been used for testing [5 iterations], c) provide the accuracy for each iteration, and d) calculate the average accuracy and the standard deviation of the accuracy. You can use the functionality for k-fold cross validation available in scikit-learn [1].

Your network is expected to achieve an average of 75% accuracy after cross validation, together with a small standard deviation.

DELIVERY INSTRUCTIONS:

- Delivery date: January 11, 2026. You have to deliver the notebook (.ipynb file) and the corresponding PDF file inside a ZIP file.
- Provide the results requested and suitable comments in the source code.
- This work can be done in groups of two people or individually (send me a message, specifying the group members, to request a group number; the list will be available in the course web page).
- IMPORTANT NOTICE: An excessive similarity between the reports/source code released can be considered a kind of plagiarism.

[1] `https://scikit-learn.org/stable/modules/model_evaluation.html`